

ARULECON: Agentic Security Rule Conversion

Ming Xu[#]
National University of
Singapore
Singapore, Singapore
mingxu@nus.edu.sg

Hongtai Wang
National University of
Singapore
Singapore, Singapore
e1132287@nus.edu.sg

Yanpei Guo
National University of
Singapore
Singapore, Singapore
guo.yanpei@nus.edu.sg

Zhengmin Yu
Fudan University
Shanghai, China
zmyu23@m.fudan.edu.cn

Weili Han
Fudan University
Shanghai, China
wlhan@fudan.edu.cn

Hoon Wei Lim
Cyber Special Ops-R&D,
NCS Group
Singapore, Singapore
hoonwei.lim@ncs.com.sg

Jin Song Dong
National University of
Singapore
Singapore, Singapore
dcsdjs@nus.edu.sg

Jiaheng Zhang
National University of
Singapore
Singapore, Singapore
jhzhzhang@nus.edu.sg

Abstract

Security Information and Event Management (SIEM) systems make it possible for detecting intrusion anomalies in real-time manner by their applied security rules. However, the heterogeneity of vendor-specific rules (e.g., Splunk SPL, Microsoft KQL, IBM AQL, Google YARA-L, and RSA ESA) makes cross-platform rule reuse extremely difficult, requiring deep domain knowledge for reliable conversion. As a result, an autonomous and accurate rule conversion framework can significantly lead to effort savings, preserving the value of existing rules. In this paper, we propose ARULECON, an agentic SIEM-rule conversion approach. Using ARULECON, the security professionals do not need to distill the source rules' logic and re-map it to target vendors, instead, they provide the source rules, the documentation of the target rules and ARULECON can purposely convert to the target vendors without more intervention. To achieve this, ARULECON is equipped with conversion intermediate representation that aligns core detection logic into vendor-neutral layer, agentic RAG pipeline that retrieves authoritative official vendor documentation to address the convention/schema mismatches, and Python-based consistency check that running both source and target rules in controlled test environments to mitigate subtle semantic drifts. We present a comprehensive evaluation of ARULECON ranging from textual alignment and the execution success, showcasing ARULECON can convert rules with higher fidelity, outperforming the baseline LLM models by 15% averagely. Finally, we perform case studies and interview with our industry collaborators in Singtel Singapore, which showcases that ARULECON can significantly save expert's time on understanding cross-SIEM's documentation and remapping logic.

CCS Concepts

• **Security and privacy** → **Software and application security**;

[#] Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792458>

Keywords

Rule-based Intrusion Detection, Agentic AI, AIOps

ACM Reference Format:

Ming Xu[#], Hongtai Wang, Yanpei Guo, Zhengmin Yu, Weili Han, Hoon Wei Lim, Jin Song Dong, and Jiaheng Zhang. 2026. ARULECON: Agentic Security Rule Conversion. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792458>

1 Introduction

Every year, approximately trillions of web intrusion attempts are made globally [34], posing severe threats to web-facing and cloud infrastructures. Security Operation Centers (SOCs) serve as front-line defense, tasked with continuously monitoring and responding to these threats in real time. While numerous studies show that neural-network-based provenance graphs [7, 12, 36] achieve high detection accuracy, however, their widespread adoption is hindered by computational complexity, expensive training costs, and a lack of interpretability. In contrast, most SOCs rely on Security Information and Event Management (SIEM) platforms [6, 35], which aggregate, filters and alert web and system logs to provide situational awareness in real-time. Prominent SIEM platforms including Splunk [32], Microsoft Sentinel [20], Google Chronicle [10], typically detect malicious activities (e.g., brute-force login) via the inherent SIEM-rules executed by their underlying analysis engines.

However, the effectiveness of SIEM platforms is tightly coupled with rules, which are a SQL-like constraint, that drive their analysis engines. In practice, organizations often undergo platform migration [29], mergers [29] and acquisitions, or operate multiple SIEM vendors in parallel. Such transitions render existing rules incompatible, as each SIEM adopts its own proprietary rules with distinct syntax, semantics, and schema requirements. Rule conversion can be performed manually by security experts, which are slow and imposes a heavy workload. Besides, organizations can rely on static vendor-provided tools, such as Microsoft-provided SPL2KQL [13], which supports a narrow conversion from Splunk to Microsoft Sentinel, leaving other platforms unsupported. Recently, they can also involve leveraging the large language models (LLMs) through prompt engineering for rule converter; however, this typically yield a poor accuracy and lacks vendor-specific correctness

due to the less exposure of SIEM's corpora of an LLM. These shortcomings call for a scalable, vendor-neutral, and reliable SIEM-rule conversion framework that retains existing rule value and eases SOC workloads.

The recent surge in the capabilities of agentic LLM-based workflows in code translation [17], SQL translation [23, 42] and software comprehension [8, 18, 39] with generative models like the GPT series, presents a paradigm-shifting opportunity for security rule conversion. Unfortunately, compared to SQL translation, SIEM-specific rule conversion is significantly more challenging due to the subtle and nuanced nature. For example, SQL has a well-defined standard that most cases follow with minor syntactic differences, while SIEM rule languages such as Splunk SPL, Microsoft KQL, IBM AQL, Google YARA-L, RSA ESA lack a unified specification. Each introduces proprietary operators, platform-specific constructs and unique data semantics tied to the vendor's logging pipeline. This heterogeneity makes rules syntactically different and semantically divergent, where equivalent operators, aggregations, or correlation logic may behave inconsistently across systems. For example, Microsoft KQL's project operator maps to Splunk SPL's fields. As a result, rule conversion requires deeper reasoning about execution semantics and domain-specific understandings, going far beyond the scope of SQL dialect translation with the challenges below:

- **Heterogeneous rule syntax across SIEM platforms.** Each platform, such as Splunk, Microsoft Sentinel, IBM QRadar, Google Chronicle and RSA NetWitness, adopts proprietary query languages, event models, and detection constructs, lacking standardized representations. As a result, direct rule migration is error-prone, with subtle differences in field naming, operators, and aggregation semantics leading to functional inconsistencies. Traditional static or general-LLM-agent-based translation approaches often fail to capture these nuances, leaving gaps in detection coverage. This motivates the design for a conversion intermediate representation (IR) that aligns core detection logic into vendor-neutral layer, ensuring consistency while enabling systematic transformation into diverse target rule formats.

- **Guaranteeing completeness and vendor-specific correctness:** LLMs are trained on general-purpose code (e.g., SQL, Python) and have limited exposure to SIEM-rule languages. Consequently, naive translations often exhibit structural omissions (e.g., missing mandatory sections/fields), syntax errors (e.g., invalid operators, clause ordering, parameter shapes), and incorrect keyword usage (e.g., misapplied macros/functions, wrong field paths or namespaces) when targeting different SIEM vendors. However, the core challenge here is not merely a matter of "syntactic translation" that enforces equivalence with the source rule, but in performing semantic mapping and knowledge supplementation to ensure the converted rule is complete and conforms to the vendor's grammar and conventions. To address this knowledge gap, we employ an agentic retrieval-augmented generation (Agentic-RAG) pipeline that dynamically retrieves authoritative official vendor documentation during generation, using it to enforce template compliance, validate allowed keywords/operators, normalize field references, and fill in required metadata-thereby improving format completeness and syntax correctness of the converted rules.

- **Verifying functional consistency through executable testing:** A further challenge arises in verifying whether the translated rules are functionally consistent with the source rules. Manually validating detection behavior against real logs is costly and slow, particularly when dealing with nested conditions, custom macros, or event aggregation. To overcome this, we propose generating Python-based executable code blocks that simulate rule execution. By synthesizing representative log data and running both source and target rules in controlled test environments, the system can check whether they yield equivalent outputs. This automated validation loop provides a concrete measure of correctness and supports iterative optimization of translated rules.

In this paper, we address the above problems by proposing ARULECON, an agentic framework for autonomous rule conversion between heterogeneous SIEM vendors. ARULECON distills the rule's core logic into a vendor-agnostic layer of conversion intermediate representation (IR), which can be converted into the target rule draft by LLM's reasoning capabilities. Then, two autonomous reflection agents including Agentic-RAG pipeline and Python-based consistency check are introduced to dynamically correct the draft rules, ensuring faithful conversion between keyword remapping, logic preserving, and functional equivalence. Running ARULECON upon 1,492 pairs of rule conversion across five mainstream SIEM platforms, results show that ARULECON consistently improves logic alignment, semantic validity, and execution success, outperforming general LLMs by around 15% in similarity alignment. Finally, we perform case studies with our industry partners, and show that ARULECON can greatly free SOC professionals from the burden of manually piecing together the cross-SIEM's documentation, and logic equivalence re-mapping during our development in operational environments.

Contributions. Our contributions are summarized below.

- We propose ARULECON, the first framework for efficient cross-SIEM rule conversion. We systematically analyze rules from multiple SIEM systems and derive a conversion Intermediate Representation (IR). In addition, we introduce two agentic reflection mechanisms that enable ARULECON to dynamically refine structural, syntactic, and semantic nuances during conversion.
- We conduct a comprehensive evaluation of ARULECON, using models of GPT-5, DeepSeek-V3, LLaMa-3. ARULECON consistently outperforms general baselines across all models, SIEM platforms and evaluation metrics.
- We uncover several empirical domain-specific rule conversion case studies in real-world deployment, which demystify the underlying domain and explain why ARULECON achieves superior performance.

We release the source codes ¹ for community development, while the prototype is being commercialized by our industry partners.

2 Background and Motivation

2.1 A Tour of SIEM and Its Applied Rules

Rule-based anomaly detection searches the predefined rule logic via the Security Information and Event Management (SIEM) [6, 35] platforms' analysis engine, which provides APIs to pass the rule

¹<https://github.com/LLM4SOC-Topic/ARuleCon>

queries to the underlying parsing and indexing engine, returning detection results such as matched logs (i.e., SQL injection, Cross-site Scripting, or Remote File Inclusion). At the core, these rules perform pattern matching on fields (e.g., IP addresses, status codes, command strings) and leverage statistical thresholds (e.g., repeated login failures within a fixed time window) to capture abnormal behaviors. Advanced rules perform temporal correlation across multiple events, linking together seemingly benign actions into a potential attack sequence, while contextual enrichment (e.g., cross-checking against threat intelligence feeds or user behavior baselines) further enhances accuracy. Unlike black-box machine learning approaches, rule-based detection provides interpretable and instant detections, allowing analysts to quickly understand the rationale and respond effectively in real time. As shown in Table 1, query-based languages such as Splunk’s SPL, Microsoft Sentinel’s KQL, and IBM QRadar’s AQL express detections as dataflow pipelines over tabular events—combining filters, projections, joins, aggregations, and time binning to suspicious patterns. Pattern-matching languages such as Google Chronicle’s YARA-L adopt a more declarative style, describing conditions and (optionally) ordered event sequences over a unified data model, which favors concise, reusable matching logic for threat hunting. RSA NetWitness ESA encodes temporal patterns and stateful correlations directly over event streams.

Table 1: Mainstream Business SIEM-rules.

SIEM-Rules	Platforms	Type
Search Processing Language (SPL)	Splunk	query-based
Kusto Query Language (KQL)	Microsoft Sentinel	query-based
Ariel Query Language (AQL)	IBM QRadar	query-based
YARA-L	Google Chronicle	pattern-based
Event Stream Analysis (ESA)	RSA NetWitness	pattern-based

Motivating Principles. Despite the their rules’ syntactic variety, these rule configurations share a common backbone: (i) predicates over normalized fields, (ii) temporal scoping, (iii) aggregation with thresholding, (iv) optional context enrichment, and (v) analyst-readable logic. The underlying logic of most rules can typically be expressed in first-order logic, which provides a common foundation for rule transformation. However, cross-platform realization of ostensibly similar logic is sensitive to differences in schema conventions, temporal semantics, string-processing semantics, operator sets, and execution models, as shown in Figure 1. Variations in normalization frameworks, windowing formalisms, evaluation order, and state management can yield non-isomorphic behavior under the same detection intent, with measurable impact on coverage, cadence, and latency. As operational environments typically evolve and migrate, portability and semantic fidelity become first concerns. Organizations frequently undergo mergers, budget adjustments, or technology upgrades that necessitate switching SIEM platforms or running multiple systems in parallel, making it essential to preserve previously crafted rules as a core SOC asset to maintain consistency and effectiveness.

2.2 Operational Observations

We empirically test conversion cases, observing that heterogeneous rule conversion is not a line-by-line or variable-to-variable mapping. For example, in Chronicle YARA-L, the aggregate operator

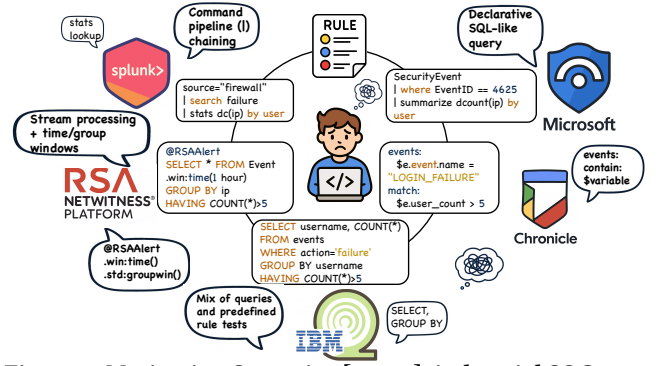


Figure 1: Motivation Scenarios [29, 40]: industrial SOCs are plagued by numerous rule query languages that share the backbones of first order logic.

is overloaded and can simultaneously express grouping, filtering, and thresholding in a single clause like:

```
events:
  filter: event.type = "login_failure"
  aggregate: count() > 5 by src_ip within 30m
```

Listing 1: This compact rule states that login failure events should be grouped by `src_ip`, restricted to a 30-minute window, and only groups with more than five failures should be retained.

A general-LLM-based conversion often attempts to map aggregate literally into QRadar AQL. Since AQL has no single operator with equivalent semantics, the model typically generates invalid or incomplete queries—for example, producing a `GROUP BY` without the threshold filter, or misplacing the condition into a simple `WHERE` clause, which changes the meaning of the rule entirely:

```
SELECT src_ip, COUNT(*)
FROM events
WHERE event_type = 'login_failure' AND COUNT(*) > 5
GROUP BY src_ip, log_source_time
```

Listing 2: The threshold `COUNT(*) > 5` is wrongly placed in the `WHERE` clause, which is syntactically invalid in AQL, and the 30-minute window constraint is ignored completely.

This motivates the design of semantic decomposition into a vendor-agnostic specification to yield a faithful conversion.

Second, syntactic operators for the same execution can differ significantly. For example, KQL’s `summarize` and `project` have no direct equivalents in SPL, where the corresponding operators are `stats` and `fields/table`. This discrepancy makes direct operator mapping by LLMs particularly challenging. This motivates the agentic probe of the vendor-specific documentation, enabling the model to consistently correct the conventions.

Third, the subtle semantic drift and functional inconsistency somewhat occurs. For example, in Splunk SPL:

```
index=auth action=failure
| stats count by src_ip
| where count > 5
```

Listing 3: This rule explicitly groups login failure events by `src_ip`, counts the number of failures for each address, and raises an alert when any single IP exceeds five attempts.

During conversion into Chronicle YARA-L, the translation collapses the semantics of the SPL pipeline into a single aggregate clause:

events:	events:
filter:	filter:
action = "failure"	action = "failure"
aggregate:	aggregate:
count() > 5	count() > 5 by src_ip

Listing 4: LLM-based Conversion **Listing 5: Faithful Conversion**

While syntactically valid, the version 4 omits the grouping by `src_ip`, effectively collapsing all failures into a global count. This failure motivates the functional consistency check between source and target rules. All these cases demonstrate that general LLM workflows/executions are insufficient and must be paired with domain-specific agentic workflows that enable semantic decomposition, targeted corrections, and adaptive reflections [28].

3 ARULECON: Methodology

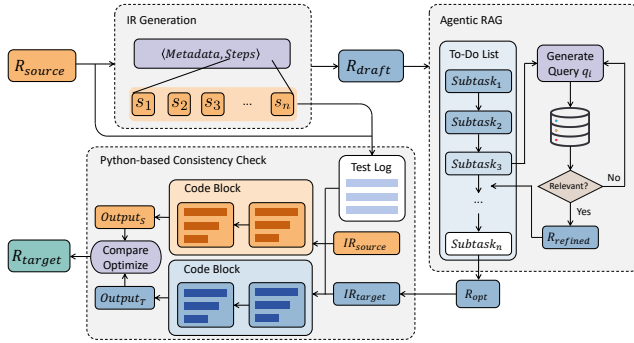


Figure 2: Overview Pipeline of ARULECON.

As shown in Figure 2, the workflow of ARULECON is structured into the interconnected generation and reflection stages. First, the source rule R_{source} is normalized into a vendor-agnostic template of intermediate representation (IR). Then, the IR is drafted into a target rule R_{draft} by the generative model, ensuring structural alignment with the grammar of the destination SIEM. Third, an Agentic RAG process refines this draft through a to-do list of subtasks, enabling iterative retrieval from vendor documentation. Then, a Python-based consistency check validates the semantic equivalence of the source and target rules by compiling their IRs into executable Python blocks, running them over synthetic test logs, and comparing outputs.

3.1 IR-driven Source Rule Interpretation

To handle the heterogeneous rule syntax, our source agent is tasked with parsing the core logic of source rules into a vendor-agnostic template of IR, enabling the model to prioritize semantic logic over syntactic details.

Intermediate Representation. To normalize SIEM rules into a unified representation, we design the IR as a layered schema that separates fundamental metadata from behavioral logic. This design abstracts away vendor-specific syntactic details while retaining the semantic fidelity of the original rule. Formally, the IR is defined as:

$$IR = \langle M, S \rangle = \langle M, \{s_1 \circ s_2 \circ \dots \circ s_n\} \rangle$$

where M denotes the set of fixed metadata fields and S represents an ordered list of steps executed sequentially. Specifically:

- $M = \{\text{rule_name}, \text{description}, \text{data_source}, \text{event_type}\}$ provides vendor-independent contextual information, ensuring that

the converted rule is properly documented and understood regardless of the underlying SIEM platform.

- $S = \{s_1 \circ s_2 \circ \dots \circ s_n\}$ captures the ordered logical flow of the rule. Each element s_i in S represents an atomic behavioral unit that the rule must preserve during conversion.

To make this precise, each step s_i is a triplet:

$$s_i = \langle \text{KEYWORD}, \text{PARAM}, \text{DESCRIPTION} \rangle$$

Where:

- **KEYWORD**: a predefined, vendor-agnostic functional abstraction over SIEM operators. A single keyword may correspond to one or multiple concrete operators across platforms (and conversely, a single vendor operator may decompose into multiple keywords). This keyword decouples semantics from syntax and enables flexible many-to-many mappings across dialects. The predefined set is: {FILTER, EXTRACT, AGGREGATE, OUTPUT, TRANSFORM, RENAME, LOOKUP, BUCKET, JOIN, FILL, APPEND, SORT, DEDUP, APPLY, DEBUG}.
- **PARAM**: the core configuration payload of the step, specifying essential arguments such as log sources, filtering predicates, time windows, grouping keys, thresholds, join keys, and other operator parameters. Importantly, we retain all parameters from the source rule in param to minimize information loss, ensuring that even subtle semantics survive the conversion process.
- **DESCRIPTION**: a concise natural-language explanation of the step's intent. This human-readable description complements the structured fields by allowing LLMs to reason about, summarize, and reorganize rule semantics across heterogeneous dialects. By embedding a semantic gloss at each step, the IR improves alignment during both initial conversion and later reflection.

For example, the YARA-L's overloaded clause (rule 1) is decomposed into multiple explicit steps like²:

```
[
  { "keyword": "filter", "param": "event.type = login_failure" },
  { "keyword": "grouping", "param": "by src_ip" },
  { "keyword": "window", "param": "within 30m" },
  { "keyword": "threshold", "param": "count() > 5" }
]
```

Through this decomposition, the IR bridges the semantic gap between Chronicle's compact but overloaded aggregate operator (rule 1) and the target rule structures. We leverage LLMs as a knowledge-grounded interpreters to parse IR, and leave the detailed prompts in Appendix B.

To ensure IR's coverage, all operator keywords across SIEM-vendors are collected manually. Unlike the IRs designed primarily for rule generation's effectiveness [38], our conversion-IR captures cross-SIEM's feasibility, enabling translation across heterogeneous vendors while maintaining semantic compactness. By explicitly capturing both keyword and param for every step, we ensure that no semantic detail from the source rule is lost during the abstraction process. Even subtle thresholds, logical operators, and time windows are preserved, providing a faithful semantic backbone for subsequent translation while maintaining core functionality.

²We omit descriptions for space optimization.

3.2 Target Rule Conversion

From the IR, a draft target rule is first generated to match the syntax and semantics of the destination platform. This draft is then refined through Agentic RAG, where documentation is retrieved and checked for consistency. To ensure functional equivalence, Python executors and synthetic logs are used to compare the outputs of source and target rules. Any mismatches are fed back into an optimization loop for repair and tuning. In this way, the process forms a cycle of generation $\rightarrow$ verification $\rightarrow$ testing $\rightarrow$ optimization, creating a closed-loop process during rule conversion.

3.2.1 Draft Generation. The first step in target rule conversion is the automatic drafting of a candidate rule for the destination SIEM platform. This stage leverages the generative capabilities of LLMs to transform the conversion representation IR into a syntactically valid rule guided by the grammar of the target vendor. To ensure both semantic fidelity and structural correctness, we adopt a prompt-engineering strategy that combines chain-of-thought decomposition with vendor-specific instructions. We leave the generation mechanism and prompts in Appendix B.

3.2.2 Agentic RAG Guidance. Then, ARULECON incorporates task planning and iterative updates driven by external knowledge. The second stage undergoes an *Agentic RAG* reflection, which refines the draft rule through retrieval and reasoning over vendor documentation. Unlike standard RAG that performs a single retrieval and passively appends passages to the prompt, Agentic RAG adopts an iterative and adaptive process. It dynamically adjusts its search queries based on the retrieved results, refining keywords until useful and precise documentation is obtained. This mimics how a human analyst consults manuals—continuously reformulating searches until the right tutorial or operator explanation is found. Such active guidance is crucial for SIEM rule conversion, where operator semantics are often subtle and not captured by a single retrieval, addressing *field mismatches*, and *under-specified operator semantics*. **Design.** The refinement of the draft rule R_{draft} is achieved through an agent-driven retrieval loop that decomposes the overall optimization into a sequence of subtasks. Given R_{draft} and the intermediate representation IR (Sec. 3.1), the system first generates an ordered to-do list

$$\mathcal{T} = \text{GenTodo}(R_{draft}, IR) = \langle t_1, \dots, t_m \rangle,$$

where each subtask t_i is defined as

$$t_i = \langle g_i, \alpha_i \rangle, \quad \alpha_i : \mathcal{P}(\mathcal{D}_V) \rightarrow \{\text{accept}, \text{reject}\}.$$

Here g_i denotes the optimization goal of the subtask, such as operator replacement, parameter adjustment, or syntax correction, while α_i is a predicate that evaluates whether a candidate evidence set from the vendor documentation $\mathcal{D}_V$ is sufficient to resolve the goal.

Each subtask is executed in sequence within an isolated context to avoid information overflow. Let the initial state be $R_{draft}^{(0)} = R_{draft}$. For the i -th subtask, the agent maintains a working context

$$C_i = \{ t_i, R_{draft}^{(i-1)}, \{q_i^{(j)}\}_{j \geq 1}, \{E_i^{(j)}\}_{j \geq 1} \},$$

where $q_i^{(j)}$ denotes the j -th query generated for task t_i , and $E_i^{(j)}$ the retrieved evidence set. The loop proceeds as

$$q_i^{(j)} = \text{GenQuery}(t_i, R_{draft}^{(i-1)}, IR), \quad E_i^{(j)} = \text{Retrieve}(q_i^{(j)}, \mathcal{D}_V),$$

followed by a judgment step

$$\alpha_i(E_i^{(j)}) \in \{\text{accept}, \text{reject}\}.$$

If $\alpha_i(E_i^{(j)}) = \text{reject}$, the agent refines the query

$$q_i^{(j+1)} = \text{RefineQuery}(q_i^{(j)}, t_i, E_i^{(j)}),$$

and repeats retrieval until acceptance is achieved. Once a satisfactory evidence set $E_i^{(j^*)}$ is accepted, it is directly used to optimize the rule. The update step is defined as

$$R_{draft}^{(i)} = \text{Apply}(R_{draft}^{(i-1)}, E_i^{(j^*)}),$$

where $E_i^{(j^*)}$ provides the vendor-grounded guidance for refinement.

Only the updated state $R_{draft}^{(i)}$ is propagated to the next subtask, ensuring that each t_i is processed independently. After all subtasks are executed in sequence, the final refined rule R_{opt} is obtained.

This design provides a clear formalism: the rule refinement is driven by a sequence of tasks $\mathcal{T}$, each resolved through iterative query generation, retrieval, judgment, and patching, with vendor documentation $\mathcal{D}_V$ serving as the authoritative source of evidence. The decomposition of optimization into a to-do list $\mathcal{T}$ prevents uncontrolled context growth: each subtask is executed within its own isolated context, avoiding information/memory overflow and reducing the risk of forgetting earlier evidence.

Our designed agentic RAG loop relies on vendor documentation as the authoritative knowledge source $\mathcal{D}_V$. For each supported SIEM platform, official manuals and rule syntax references are collected. To support fine-grained retrieval, each corpus is pre-processed by extracting its table of contents to build a hierarchical index of supported operators and functions. This structure provides the retrieval agent with coarse-grained entry points to relevant sections. To improve retrieval precision beyond naive keyword matching, a vendor-specific query set $Q_V = \{q_1, \dots, q_p\}$ is constructed, where each query is annotated with its functional category (e.g., *filtering*, *aggregation*, *temporal window*, *join*, *lookup*). During task execution, the agent dynamically selects or generates queries from Q_V to retrieve documentation fragments most relevant to the current optimization goal. The retrieved passages are then stored as evidence sets E_i , which directly support rule updates in the refinement loop.

3.2.3 Python-based Consistency Check. To fix the subtle semantic drift problems, the final stage of the conversion loop focuses on verifying the semantic consistency between the source rule $Rule_S$ and the converted rule $Rule_T$. To achieve this, we employ a Python-based execution framework that simulates the behavior of SIEM rules under controlled conditions. Test logs $\mathcal{L}$ are generated, consisting of benign events $\mathcal{L}_{\text{normal}}$ and abnormal events $\mathcal{L}_{\text{attack}}$, ensuring that both rules are evaluated under realistic scenarios.

As summarized in Algorithm 1, the converted rule $Rule_T$ is then normalized into a conversion representation IR_T , so that both source and target rules are expressed as IRs in a comparable format. A pipeline executor processes each IR step $s_i \in IR$ sequentially: for every step, Python code is generated and executed, producing an output that becomes the input for the next step. In this way, the entire IR functions as a compositional pipeline $f_1 \circ f_2 \circ \dots \circ f_n$ applied to $\mathcal{L}$. After both pipelines are executed, the outputs O_S from IR_S and O_T from IR_T are compared. Any mismatches $\Delta = O_S \Delta O_T$

are analyzed to identify issues such as missing filters, threshold misalignments, or aggregation discrepancies, which then serve as basis for generating optimizations to refine the target rule.

Algorithm 1 Python-based Consistency Check

Require: Source rule $Rules_S$, source IR IR_S , target rule $Rule_T$
Ensure: Semantic equivalence score and optimization suggestions

```

1:  $\mathcal{L} \leftarrow \text{GenerateTestLogs}(IR_S, Rules_S)$  /*Generate Test Logs*/
2:  $IR_T \leftarrow \text{DeriveIR}(Rule_T)$  /*Derive Target IR*/
3: for each  $IR \in \{IR_S, IR_T\}$  do
4:    $data \leftarrow \mathcal{L}$ 
5:   for each step  $s_i = \langle keyword, param, description \rangle$  in  $IR$  do
6:      $code \leftarrow \text{GeneratePython}(s_i)$ 
7:      $output \leftarrow \text{Run}(code, data)$ 
8:      $data \leftarrow output$  {feed into next step}
9:   end for
10:   $O[IR] \leftarrow data$ 
11: end for
12:  $O_S \leftarrow O[IR_S]$ ,  $O_T \leftarrow O[IR_T]$  /*Compare Outputs*/
13:  $diffs \leftarrow \text{Compare}(O_S, O_T)$ 
14:  $suggestions \leftarrow \text{OptimizeTargetRule}(diffs, \mathcal{R}_T)$  /*Optimize Target Rule*/
15: return  $suggestions$ 

```

Take the source SPL rule 3 and the target YARA-L rule 4 as an example, to verify the equivalence, our system generates executable Python functions below. Each function takes a list of dictionaries (representing logs) as input, executes the rule logic, and returns the result. The source executor correctly outputs ['1.2.3.4'], while the naïve target incorrectly returns a global match ['ALL']. The mismatch is flagged, prompting the system to add the missing grouping, which yields the corrected target rule.

```

from typing import List, Dict
test_log = [
    {"action": "failure", "src_ip": "1.2.3.4"},
    {"action": "failure", "src_ip": "1.2.3.4"},
    ...
    {"action": "failure", "src_ip": "5.6.7.8"}]
def exec_source_rule(logs: List[Dict]) -> List[str]:
    # SPL semantics: count failures per src_ip, keep those
    # > 5
    df = pd.DataFrame(logs)
    d = df[df["action"] == "failure"]
    grp =
    # d.groupby("src_ip").size().reset_index(name="cnt")
    return sorted(grp.loc[grp["cnt"] > 5,
    # "src_ip"].tolist())
def exec_target_rule(logs: List[Dict]) -> List[str]:
    # Naïve YARA-L semantics: global count only (incorrect)
    df = pd.DataFrame(logs)
    d = df[df["action"] == "failure"]
    return ["ALL"] if len(d) > 5 else []

```

The Python-based Consistency Check serves as the final safeguard in the conversion pipeline by verifying the intended detection functions. Unlike purely syntactic comparison, this stage evaluates rules through actual execution over synthesized test logs $\mathcal{L}$, ensuring that logical operators, thresholds, and aggregations behave consistently across heterogeneous SIEM platforms. By modeling each IR step s_i as an executable Python function and propagating intermediate results $data_i$ sequentially, the framework exposes subtle inconsistencies that may not be visible at the textual level, such as missing filters or altered evaluation windows. The comparison of

outputs O_S and O_T highlights semantic gaps Δ that guide targeted refinements, allowing the system to generate concrete optimization suggestions for $Rule_T$.

4 Evaluation

We quantitatively evaluate the following research questions (RQs): RQ1–**Conversion Accuracy**: How effective is ARULECON in converting a source rule into the target rule? RQ2–**Ablation Studies**: How does each component of ARULECON contribute to the overall performance? RQ3–**Efficiency Overhead**: What are the latency and costs during the rule conversion process?

4.1 Experimental Settings

4.1.1 Model and Parameters. We build ARULECON upon three state-of-the-art LLMs: GPT-5, DeepSeek-V3 (671B), and LLaMA-3 (405B). The DeepSeek-V3 and LLaMa-3 models are downloaded from Hugging Face [3], [1]. To control generation behavior, all models are configured with a temperature of 0.3 (balancing determinism and flexibility), top-p of 0.9 (ensuring controlled diversity), and a maximum response length of 1024 tokens to prevent excessively long outputs. Parameters used in Agentic RAG and Python-based consistency check are shown in Appendix C.1.

4.1.2 Datasets and Setups. Toward evaluating the conversion quality across heterogeneous SIEM rules, we collect datasets from five widely-used platforms. All datasets are constructed from their official websites or open-sourced repositories, and each rule entry includes the rule body with a natural language description explaining its purpose and application context. The statistics of these datasets are summarized in Table 2. The number of rules varies significantly across vendors, ranging from only a few dozen in IBM QRadar and RSA NetWitness to over one thousand in Splunk. To avoid evaluation bias caused by this imbalance, we randomly sample approximately 100 rules from each dataset when the total size exceeds this threshold. We sequentially use each vendor as the source and convert its rules into the remaining four target vendors, yielding 1,492 conversion pairs across five SIEMs.

Table 2: Summary of SIEM-rule datasets.

SIEM-Rules	Size (Used)	Time	Focus/Coverage
Splunk SPL [33]	1725 (100)	2025	Cloud, Client-Side
Microsoft KQL [22]	483 (100)	2025	ASimDNS, ASimFileEvent
IBM AQL [14]	33	2019	General Security Events
Google YARA-L [11]	348 (100)	2025	AWS, Google Cloud Platform, Github
RSA ESA [26]	40	2018	Web and Network Detection

4.1.3 Metrics. We use the structural and lexical alignment between the converted rules and the source rules as an objective accuracy measurement as below.

- **CodeBLEU** [25]. To evaluate structural fidelity, we follow prior work and adopt CodeBLEU, a code-oriented extension of BLEU. Instead of comparing surface-level tokens alone, CodeBLEU incorporates abstract syntax tree (AST) matching, data-flow consistency, and weighted n-gram matching to capture both lexical and syntactic similarity. In our setting, rules from different vendors are

first converted into Python-equivalent code through a uniform prompting template, and CodeBLEU is then computed between the generated code and the reference code. This allows us to assess whether the converted rule preserves the same computational logic as the source rules, independent of vendor-specific syntax.

- **Embedding Similarity.** To complement structural comparison, we compute semantic similarity between rules by embedding them into a continuous vector space using text-embedding-ada-002 [2]. Cosine similarity is then calculated between the generated rule and the source rule. This approach captures semantic alignment even when the rules differ in keywords or ordering, which is reasonable given that SIEM rules often exhibit dialectal variations but convey the same detection logic.

- **Logic Slot Consistency.** Beyond text or vector similarity, we measure whether the essential logical slots are preserved. Each rule is decomposed into a set of semantic slots, including predicates, temporal windows, aggregation functions, join conditions, and threshold expressions. We extract these slots via regular-expression templates for each SIEM dialect, normalize values (e.g., thresholds and time units), and compute slot-level similarity using token overlap and Jaccard-based matching. The metric compares these slots directly, ignoring surface syntactic differences, and reports the proportion of matched slots between the generated and source rules. This provides a fine-grained view of logical equivalence, ensuring that even when keywords diverge, the underlying detection semantic remains intact.

We have tested that the similarity measurements correlate positively with functional equivalence, making them lightweight quantification proxies: In our sampled functionally-equivalent conversions (Appendix C.2), pairs typically score higher on CodeBLEU (8/10), Embedding Similarity (7/10), and Logic Slot Consistency (10/10). To show the functional equivalence between source and target rules, we also employ six semantic-fidelity metrics with an LLM-as-a-judge paradigm in Appendix C.2, avoiding the syntactically similar yet semantically divergent phenomenon.

Execution Success. We use Python-based parsers [4, 30, 37] and open-source validation libraries to test whether the converted rules can be executed upon the target SIEMs. Particularly, we implement grammar-level checks and structural validation routines that verify keyword ordering, clause nesting, and aggregation syntax for formats such as SPL, KQL, and AQL.

4.1.4 Baseline. We compare ARULECON’s performance against a customized LLM based upon the LLMs of GPT-5, DeepSeek-V3, and LLaMa-3, without the intermediate representation, agentic RAG and Python-based reflection mechanisms. The baselines generate rules using the same prompts as those employed by ARULECON (shown in Table 6). Both ARULECON and baseline are under the same parameter settings for fair comparison.

4.2 Experimental Results

4.2.1 RQ1-Accuracy. We present our *quantifiable accuracy assessment* in Table 3. ARULECON (AC) consistently outperforms the baselines (BL) across all models, SIEM platforms, and evaluation metrics. Overall, GPT achieves the strongest absolute performance, while DeepSeek and LLaMA also benefit substantially from ARULECON, confirming that the improvements are model-agnostic. Conversions involving Splunk and RSA NetWitness generally yield higher logic

slot similarity, whereas IBM QRadar conversions are relatively more challenging when preserving their logic slot consistency. This is perhaps due to QRadar’s stricter separation of stateless vs. stateful tests, complex condition ordering, and performance penalties associated with regex parsing.

On aggregate, ARULECON delivers consistent gains across all three metrics. For GPT, the average improvement over BL is +9.1% in CodeBLEU, +10.3% in embedding similarity, and +11.6% in logic slot consistency. DeepSeek shows gains of +8.4%, +11.1%, and +13.0% on the same metrics, while LLaMA improves by +9.7%, +10.8%, and +12.1% respectively. When examining the full range, the relative improvements vary between +5% and +15% depending on the conversion direction and metric. These results suggest that ARULECON enhances surface-level similarity, strengthens logic preservation as reflected by the larger margins on embedding similarity and logic slot consistency. We have tested ARULECON can also outperform the code-execution agents (i.e., OpenHands, SWE-Agent), which use generic, tool-heavy workflows, and lack the task-specific iterative refinement for precise vendor-specific grammar adaption/subtle semantic-reformatting, always yielding poor performance.

Failure Cases. To grasp the underlying conversion boundaries, we analyze the common difficulties like nested query structures, and rules involving temporal windows and thresholds in Appendix C.3. We manually check the lower-ranked cases of source-target conversions and reveals that the failure is typically caused by the SIEM-specific characteristics, as detailed below:

- **Stateful vs. stateless rule evaluation in IBM QRadar.** QRadar distinguishes between stateless single-event tests and stateful multi-event correlation. Conversion often fails when stateful conditions (e.g., repeated failures within a window) are flattened into stateless filters in target SIEMs, losing the temporal correlation.
- **Temporal and aggregation mismatches in Microsoft Sentinel and Google Chronicle.** Sentinel KQL and Chronicle rules support long-range queries, flexible time binning, and large-scale historical aggregation. These constructs may not be supported in platforms with stricter time windows or limited aggregation functions, causing incomplete or invalid translations.

Execution Success. We report the execution validity of the converted rules across heterogeneous SIEM vendors in Table 4, presenting results for ARULECON while omitting baseline results due to their significant poor performance. Overall, ARULECON achieves high syntax validity rates, with most conversions reaching above 90%. In particular, conversions involving Google Chronicle and Splunk tend to achieve near-perfect success (close to 100%). This is likely because these two platforms not only have greater market adoption, which increases the likelihood that LLMs were exposed to their syntax during training, but also provide clearer and more structured official documentation, enabling models to better capture execution nuances. In contrast, conversions targeting IBM QRadar and RSA NetWitness occasionally fall below full validity, with scores around 92–97%. Their weaker performance can be explained by more complex grammar rules, combined with limited publicly available datasets and less comprehensive documentation, which together hinder the LLM’s ability to generalize. Despite these challenges, the results confirm that ARULECON remains lightweight yet reliable in syntactic verification.

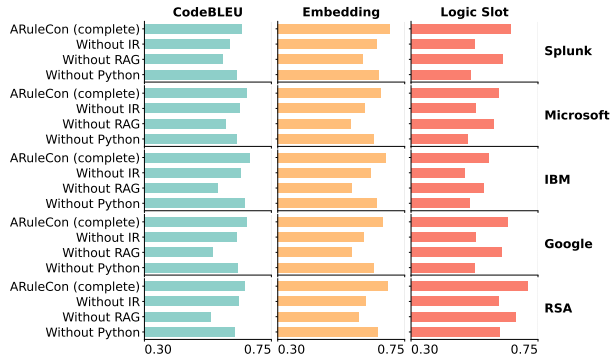
Table 3: Similarity comparison between the converted rules and source rules from ARULECON (AC) and baselines (BL).

Source SIEM → Target SIEM	GPT-5		CodeBLEU (↑)				Embedding Similarity (↑)						Logic Slot Consistency (↑)					
	AC	BL	DeepSeek-V3	AC	BL	LLaMa-3	AC	BL	DeepSeek-V3	AC	BL	LLaMa-3	AC	BL	DeepSeek-V3	AC	BL	LLaMa-3
Splunk → Microsoft Sentinel	63.7	58.9	60.5	55.4	58.6	52.9	69.3	63.8	66.1	60.3	62.7	57.1	73.2	67.6	69.4	63.8	66.2	60.9
Splunk → IBM QRadar	63.4	58.2	61.1	55.9	59.0	53.5	64.1	58.6	62.3	56.9	60.4	54.8	65.5	60.2	63.1	57.7	60.8	55.3
Splunk → Google Chronicle	63.1	58.0	60.2	55.0	58.1	53.1	83.6	77.9	80.2	74.7	77.5	71.8	68.0	62.6	65.1	59.6	62.9	57.4
Splunk → RSA NetWitness	62.8	57.6	60.0	55.1	58.2	53.0	73.2	67.9	70.4	65.2	67.1	61.9	68.4	63.1	65.7	60.5	62.6	57.2
Microsoft Sentinel → Splunk	63.7	58.6	61.0	55.8	58.9	53.6	65.2	59.8	62.7	57.4	60.1	54.9	70.9	65.4	67.8	62.3	64.2	58.8
Microsoft Sentinel → IBM QRadar	67.4	61.8	64.2	58.9	61.1	55.7	54.6	49.4	52.9	47.6	50.3	45.2	61.3	56.0	59.2	53.7	56.5	51.1
Microsoft Sentinel → Google Chronicle	67.2	61.6	64.0	58.7	61.0	55.6	75.8	70.3	73.1	67.7	70.2	64.9	62.1	56.8	59.7	54.5	57.1	51.9
Microsoft Sentinel → RSA NetWitness	67.9	62.5	64.6	59.4	61.6	56.1	79.7	74.2	76.5	71.0	73.3	68.0	64.4	59.0	61.7	56.3	58.6	53.7
IBM QRadar → Splunk	67.9	62.6	64.7	59.2	61.8	56.4	70.3	65.1	67.2	62.0	64.1	59.0	52.2	47.1	50.4	45.2	48.1	43.0
IBM QRadar → Microsoft Sentinel	67.5	62.1	64.4	59.0	61.2	56.0	72.5	67.3	69.3	64.2	66.2	61.0	57.4	52.3	55.1	50.0	52.6	47.5
IBM QRadar → Google Chronicle	69.1	63.6	65.9	60.5	62.8	57.3	82.4	76.8	79.3	74.0	76.1	70.6	56.9	51.8	54.7	49.6	52.1	47.1
IBM QRadar → RSA NetWitness	68.9	63.3	65.7	60.2	62.6	57.1	81.9	74.2	76.5	73.6	73.3	68.1	56.6	51.5	54.3	49.3	51.7	46.8
Google Chronicle → Splunk	62.0	56.8	59.4	54.1	57.2	52.1	65.5	63.3	66.0	60.7	63.4	58.2	57.6	52.4	55.1	50.0	52.7	47.6
Google Chronicle → Microsoft Sentinel	64.6	59.3	61.7	56.6	59.1	54.1	64.7	59.6	62.0	57.0	59.4	54.4	62.5	57.2	60.1	54.9	57.4	52.3
Google Chronicle → IBM QRadar	63.2	58.0	60.6	55.4	58.4	53.1	63.2	58.1	60.8	55.7	58.6	53.6	52.1	47.0	50.2	45.2	47.9	43.0
Google Chronicle → RSA NetWitness	65.1	59.9	62.0	56.9	59.6	54.4	89.6	84.3	86.0	80.9	83.4	78.1	75.2	69.8	72.5	67.2	69.7	64.4
RSA NetWitness → Splunk	69.7	64.2	66.3	61.1	63.1	57.7	63.0	57.9	60.4	55.4	57.8	52.6	74.1	68.8	71.3	66.1	68.6	63.4
RSA NetWitness → Microsoft Sentinel	68.3	62.8	65.1	59.7	62.0	56.6	62.3	57.3	59.8	54.7	57.0	52.0	72.3	67.0	69.6	64.3	66.7	61.6
RSA NetWitness → IBM QRadar	71.5	66.0	68.2	62.9	65.3	60.0	59.9	54.7	57.6	52.4	55.1	50.1	81.1	75.8	78.2	72.9	75.4	70.2
RSA NetWitness → Google Chronicle	67.2	61.7	64.1	58.8	61.0	55.9	64.1	59.0	61.4	56.4	58.8	53.8	78.9	73.4	75.9	70.6	73.1	67.8

Table 4: Execution success rate on the target SIEM Platforms.

Source Rules	Target Rules	Splunk	Microsoft Sentinel	IBM QRadar	Google Chronicle	RSA NetWitness
Splunk	–	–	0.868	0.774	1.000	0.868
Microsoft Sentinel	1.000	–	–	1.000	1.000	0.943
IBM QRadar	1.000	0.970	–	–	1.000	0.939
Google Chronicle	1.000	1.000	1.000	–	–	1.000
RSA NetWitness	1.000	0.925	1.000	1.000	–	–

4.2.2 RQ2-Ablation Study. The results shown in Figure 3 present the ablation study to assess the necessity of each component. Without **IR**, the model loses **fine-grained logical consistency**, since compact operators (e.g., aggregate, tstats) are harder to decompose without structured steps. Excluding **Agentic RAG** could lead to lower **embedding similarity**, as vendor-specific syntax often requires additional documentation support for accurate operator selection. Removing the **Python-based consistency check** appears to reduce **logic slot alignment**, as execution validation helps identify subtle issues such as missing GROUP BY or misplaced thresholds.

**Figure 3: Each component contributes to the performance.**

4.2.3 RQ3-Efficiency. We present the computational and economic costs in Table 5, which are derived by running ARULECON and a

Table 5: Efficiency and cost of ARULECON and baselines.

Model		Prompt Tokens	Output Tokens	Money Cost (USD)	Generation Time (s)
GPT-5	ARULECON	20,184	3,042	0.046	142
	Baseline	2,137	514	0.008	12
DeepSeek-V3	ARULECON	18,532	2,874	–	177
	Baseline	1,954	472	–	16
LLaMA-3	ARULECON	16,947	2,601	–	163
	Baseline	1,806	438	–	18

baseline direct translation on five SIEM rule dialect datasets, and averaging the results across multiple test cases. We observe that ARULECON requires more tokens and computation time than the baseline, mainly due to the multi-step reflection. The main reason could be the optimization steps: 1) the agentic RAG process requires multiple rounds of retrieval to select the most relevant vendor documentation; 2) the generated Python code occasionally contains errors that must be fixed through iterative debugging. However, as SIEM-rule translation is a high-stake security task, where 15%-accuracy-gain justifies an additional 100-seconds of computation. The misconverted rules translate into missed detections, which can propagate into downstream incident-response failures.

5 Conclusion and Future Works

In this paper, we propose ARULECON, an agentic rule conversion framework. By leveraging the intermediate representation and two reflection agents, ARULECON can reliably translate a source SIEM-rule into the target. Our case study with industry collaborators shows that ARULECON can help industry practices in their operational workflow by lowering the progress of consulting documentation and rewriting queries. Future work can collect a larger corpus of paired source-target rules to fine-tune and align the models, further improving accuracy.

Acknowledgments

This paper is supported by the Minister of Education, Singapore (MOE-T2EP20125-0015), and the NUS-NCS Joint Laboratory for Cyber Security backed up by Singtel Singapore.

References

- [1] 2022. meta-llama/llama-3.1-405B · Hugging Face — huggingface.co. <https://huggingface.co/meta-llama/llama-3.1-405B>.
- [2] 2022. text-embedding-ada-002. <https://platform.openai.com/docs/guides/embeddings/what-are-embeddings>.
- [3] 2024. deepseek-ai/DeepSeek-V3 · Hugging Face — huggingface.co. <https://huggingface.co/deepseek-ai/DeepSeek-V3>.
- [4] Andi Albrecht and contributors. [n.d.]. sqlparse: non-validating SQL parser for Python. <https://pypi.org/project/sqlparse/>.
- [5] Ariel Query Language Guide [n.d.]. https://www.ibm.com/docs/en/SS42VS_7.4/pdf/b_qradar_aql.pdf. Ariel Query Language Guide.
- [6] Sandeep N. Bhatt, Pratyusa K. Manadhata, and Loai Zomlot. 2014. The Operational Role of Security Information and Event Management Systems. *IEEE Secur. Priv.* 12, 5 (2014), 35–41. doi:10.1109/MSP.2014.103
- [7] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2023. Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance. *CoRR abs/2308.05034* (2023). arXiv:2308.05034 doi:10.48550/ARXIV.2308.05034
- [8] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16–20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 1536–1547. doi:10.18653/V1/2020.FINDINGS-EMNLP.139
- [9] Google Cloud [n.d.]. <https://cloud.google.com/chronicle/docs/detection/yara-l-2-0-syntax>. YARA-L 2.0 language syntax.
- [10] Google Security Operations (SecOps) [n.d.]. <https://cloud.google.com/security/products/security-operations?hl=en>. Google SecOps' cloud-native security operations platform empowers security teams to better detect, investigate, and respond to cybersecurity threats..
- [11] Google YARA-L Rules [n.d.]. <https://github.com/chronicle/detection-rules/tree/main/rules/community>. Google YARA-L Rule Dataset.
- [12] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical Provenance Analysis for Endpoint Detection and Response Systems. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18–21, 2020*. IEEE, 1172–1189. doi:10.1109/SP40000.2020.00096
- [13] IBM. 2024. SPL To KQL Application.
- [14] IBM AQL Rules [n.d.]. <https://github.com/Xboarder56/QRCE-Rules>. IBM QRadar AQL Rule Dataset.
- [15] Cohen's Kappa. [n.d.]. A measure of agreement between two dependent categorical samples. <https://datatab.net/tutorial/cohens-kappa>.
- [16] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. arXiv:2412.05579 [cs.CL] <https://arxiv.org/abs/2412.05579>
- [17] Ruishi Li, Bo Wang, Tianyu Li, Prateek Saxena, and Ashish Kundu. 2025. Translating C To Rust: Lessons from a User Study. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24–28, 2025*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/translating-c-to-rust-lessons-from-a-user-study/>
- [18] Jiawei Liu, Nirav Diwan, Zhe Wang, Haoyu Zhai, Xiaona Zhou, Kiet A. Nguyen, Tianjiao Yu, Muntasir Wahed, Yinlin Deng, Hadjer Benkraouda, Yuxiang Wei, Lingming Zhang, Ismini Lourentzou, and Gang Wang. 2025. PurpCode: Reasoning for Safer Code Generation. arXiv:2507.19060 [cs.CR] <https://arxiv.org/abs/2507.19060>
- [19] Zhengxiong Luo, Huan Zhao, Dylan Wolff, Cristian Cadar, and Abhik Roychoudhury. 2026. Agentic Concolic Execution. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 1–19.
- [20] Microsoft. 2025. Microsoft Sentinel - Cloud-native SIEM Solution | Microsoft Azure — azure.microsoft.com. <https://azure.microsoft.com/en-us/products/microsoft-sentinel>.
- [21] Microsoft Ignite [n.d.]. <https://learn.microsoft.com/en-us/kusto/query/?view=microsoft-fabric>. Kusto Query Language overview.
- [22] Microsoft KQL Rules [n.d.]. <https://github.com/Azure/Azure-Sentinel/tree/master/Detections>. Microsoft Sentinel KQL Rule Dataset.
- [23] Amadou Latyr Ngom and Tim Kraska. 2024. Mallet: SQL Dialect Translation with LLM Rule Generation. In *Proceedings of the Seventh International Workshop on Exploiting Artificial Intelligence Techniques for Data Management, aiDM 2024, Santiago, Chile, 14 June 2024*. ACM, 3:1–3:5. doi:10.1145/3663742.3663973
- [24] Binhang Qi, Yun Lin, Xinyi Weng, Yuhuan Huang, Chenyan Liu, Hailong Sun, Zhi Jin, and Jin Song Dong. 2025. Intention-driven generation of project-specific test cases. *arXiv preprint arXiv:2507.20619* (2025).
- [25] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. *CoRR abs/2009.10297* (2020). arXiv:2009.10297 <https://arxiv.org/abs/2009.10297>
- [26] RSA ESA Rules [n.d.]. <https://github.com/netwitness/nw-esa>. RSA NetWitness ESA Rule Dataset.
- [27] Rule SyntaxRule Syntax [n.d.]. <https://community.netwitness.com/s/article/RuleSyntax>. NetWitness Community.
- [28] Stuart J Russell and Peter Norvig. 2016. *Artificial intelligence: a modern approach*. Pearson.
- [29] SIEM Migration [n.d.]. <https://cardinalops.com/blog/siem-migration-challenges-strategies/>. SIEM Migration: Challenges and Strategies..
- [30] Kirill Simonov and contributors. [n.d.]. PyYAML: YAML parser and emitter for Python. <https://pypi.org/project/PyYAML/>.
- [31] Splunk [n.d.]. <https://help.splunk.com/en/splunk-enterprise/search/spl-search-reference/9.4/introduction/welcome-to-the-search-reference>. Splunk Enterprise.
- [32] Splunk cisco company [n.d.]. <https://shorturl.at/dgTsP>. State of Security 2024: The Race to Harness AI.
- [33] Splunk SPL Rules [n.d.]. https://github.com/splunk/security_content/tree/develop/detections. Splunk SPL Rule Dataset.
- [34] Threat Report [n.d.]. https://www.cloudwards.net/cyber-security-statistics/?utm_source=chatgpt.com. 67 Cyber Security Statistics, Facts and Trends: Data on Attacks, Breaches and Threats for 2025.
- [35] Rafael Uetz, Marco Herzog, Louis Hackländer, Simon Schwarz, and Martin Henze. 2023. You Cannot Escape Me: Detecting Evasions of SIEM Rules in Enterprise Networks. *CoRR abs/2311.10197* (2023). arXiv:2311.10197 doi:10.48550/ARXIV.2311.10197
- [36] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *2024 IEEE Symposium on Security and Privacy (SP)*. 3552–3570. doi:10.1109/SP54263.2024.00139
- [37] VirusTotal. [n.d.]. yara-python: The Python interface for YARA. <https://pypi.org/project/yara-python/>.
- [38] Hongtai Wang, Ming Xu, Yanpei Guo, Weili Han, Hoon Wei Lim, and Jin Song Dong. 2025. RulePilot: An LLM-Powered Agent for Security Rule Generation. *CoRR abs/2511.12224* (2025). arXiv:2511.12224 doi:10.48550/ARXIV.2511.12224
- [39] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. arXiv:2502.18449 [cs.SE] <https://arxiv.org/abs/2502.18449>
- [40] Why SIEM Migration [n.d.]. DemystifyingSIEMmigration: Pitfallstoavoidandtipsforensuring success. Demystifying SIEM migration: Pitfalls to avoid and tips for ensuring success..
- [41] Pengfei Wu, Jianting Ning, Wu Luo, Xinyi Huang, and Debiao He. 2021. Exploring dynamic task loading in SGX-based distributed computing. *IEEE Transactions on Services Computing* 16, 1 (2021), 288–301.
- [42] Wei Zhou, Guoliang Li, Haoyu Wang, Yuxing Han, Xufei Wu, Fan Wu, and Xuanhe Zhou. 2025. PARROT: A Benchmark for Evaluating LLMs in Cross-System SQL Translation. arXiv:2509.23338 [cs.DB] <https://arxiv.org/abs/2509.23338>

A Discussion

Constraint Translation. Agentic AI offers a promising paradigm for constraint translation (e.g., translating the C programming language to Rust [17], or SQL translations [23, 42]), where traditional static approaches often fail to capture subtle semantics and domain-specific dependencies. ConcoLLMic [19] develops a novel approach that uses LLM agents to model symbolic execution: symbolic modeling and constraint solving. IntentionTest [24] proposes an intention-constrained approach for generating project-specific test cases. The domain specific constraints are also widely used in SGX-related domains [41].

Takeaways. ARULECON provides insights for code translation, SQL translation, and other domain-specific constraint conversions. This methodology suggests that similar layered designs can reduce semantic drift and improve executability in cross-system translation tasks beyond SIEMs.

B Prompt Mechanisms in ARULECON

IR Generation Mechanism/Prompts. To ensure reliable extraction of IR from heterogeneous source rules, we leverage LLMs not only as free-form parsers but as knowledge-grounded interpreters.

Instead of relying solely on the source rule text, we provide the model with vendor-specific tutorials that are distilled from official SIEM documentation. These tutorials summarize how filtering conditions, aggregation clauses, and temporal windows are typically expressed in each platform, serving as a lightweight guide for the LLM to align its interpretation with vendor conventions. The detailed tutorials and prompts are shown in our open-sourced code. **Draft Rule Generation Mechanism/Prompts.** The IR is serialized into a structured input containing metadata (M) and sequential steps (S), which are then injected into a vendor-aware prompt template. The template specifies the SIEM context, (e.g., “You are a security analyst specializing in Microsoft Sentinel KQL”), enumerates a task list for reasoning (e.g., mapping log sources, applying filters, performing aggregations, validating syntax), and provides few-shot exemplars where available. An example of such a template is shown in Table 6. The draft produced at this stage preserves the semantics captured in $\mathcal{R}$ while adhering to the structural requirements of the target vendor. Although not guaranteed to be fully correct, this draft provides a consistent and syntactically valid baseline for subsequent refinement.

Table 6: Structure of the Draft Generation Prompt

Vendor-Aware CoT Prompt Template
System Role: You are a security analyst at a cybersecurity company, specializing in writing and optimizing <Target SIEM> rules for threat detection. Task: Convert the following conversion Intermediate Representation (IR) into a syntactically valid <Target SIEM> rule. Input (IR): Metadata M and ordered steps $S = (s_1, s_2, \dots, s_n)$, where each $s_i = \langle \text{keyword}, \text{param}, \text{description} \rangle$. Reasoning Instructions: Follow chain-of-thought reasoning to: ① Identify relevant event sources from M ② Map IR keywords to equivalent operators in <Target SIEM> ③ Apply parameters (param) precisely without loss ④ Verify temporal windows, aggregations, and thresholds ⑤ Construct a rule that is both syntactically correct and semantically faithful ⑥ Validate against vendor grammar before final output Output: A single <Target SIEM> detection rule, wrapped in code block format. Example Input: { "keyword": "AGGREGATE", "param": "count() by src_ip where action=failure", "description": "Detect source IPs with failed logins" } Example Output (KQL): SecurityEvent where ActionType == "failure" summarize count() by src_ip where count_ > 5

C Evaluation

C.1 Parameters of Components

For the Agentic RAG-based reflection component, we construct a vendor-specific retrieval corpus based on the official documentation of Splunk SPL [31], Microsoft Sentinel KQL [21], IBM QRadar AQL [5], Google Chronicle YARA-L [9], and RSA NetWitness ESA [27]. The documentation is processed through a hierarchical chunking strategy. Each document is first segmented according to its chapters or sections; if a section exceeds a predefined length threshold, it is recursively split into smaller overlapping chunks. Each chunk is then embedded using text-embedding-ada-002 [2], producing dense vector representations stored in Chroma that capture semantic similarity across heterogeneous rule descriptions. During evaluation, the retrieval agent selects the top-5 most relevant passages for each rule clause and injects them into the prompt, grounding the model’s reflection in authoritative references.

The reflection process in ARULECON is bound to ensure both effectiveness and efficiency. For the Agentic RAG component, when a

converted rule exhibits mismatched or incomplete keywords during IR extraction, the model is prompted again with retrieved documentation passages and asked to revise the specific step; this iteration is repeated at most $N = 3$ times. For the Python code generation stage, if the generated code fails execution (e.g., syntax errors or runtime mismatches), the system triggers a separate reflection loop in which the model debugs and regenerates the code, with a maximum of $K = 5$ attempts.

C.2 Semantic-fidelity Evaluation

Metrics. The quantifiable similarity assessment offers an objective metric, yet it may yield misleadingly high scores when rules are syntactically similar but semantically incorrect. To mitigate this bias, we adopt the LLM-as-a-judge paradigm, a scalable method for approximating human preferences [16] over the six key evaluation dimensions below.

- *Event Scope & Schema Mapping (E/S).* Whether the source and converted rules operate over the same event sources and fields, including index/table selection, event type alignment, and schema-level field mappings (e.g., host $\leftrightarrow$ Computer, src_ip $\leftrightarrow$ IPAddress).
- *Predicate & Boolean Logic (P/B).* Equivalence of filtering conditions, covering operators, negations, case sensitivity, regular expressions, and logical combinations of predicates, such as list membership and range boundaries.
- *Temporal Semantics & Windows (T/W).* Consistency in time-related constraints, including window size and type (tumbling vs. sliding), alignment anchors, and the placement of temporal conditions before or after aggregation.
- *Aggregation & Thresholding (A/T).* Alignment of aggregation functions, grouping keys, distinct semantics, threshold comparisons, and whether post-aggregation filtering (e.g., having) is preserved.
- *Correlation & Joins (C/J).* Correctness of multi-event or multi-source correlation, including join keys, join type, temporal constraints for co-occurrence, and directionality or deduplication strategies.
- *Alert Semantics & Outcome (A/O).* Preservation of the alerting conditions and outputs, including trigger criteria, deduplication or throttling settings, entities in the result (e.g., IP, user), and severity levels or tagging.

We adopt a scoring scheme ranging from 0 to 1 for each evaluation dimension. We use relative scores between outputs under the same prompt in the baseline and ARULECON, instead of absolute scores. To mitigate evaluation bias, we adopted a human-aligned iterative evaluation protocol, where an experienced human expert and an LLM jointly refined prompts to ensure consistent evaluation standards. We categorize each pairwise comparison into three possible outcomes: “ARuleCon better”, “Tie”, and “Baseline better”. We define inter-rater agreement as a match in relative preference; for example, both the human and the LLM preferring ARULECON over the corresponding vanilla LLMs (baseline) is considered consistent, regardless of exact numerical differences. Under this definition, the inter-rater agreement results are visualized in Figure 5. The agreement reaches a Cohen’s Kappa [15] score larger than 0.885, indicating strong consistency between human judgment and LLM-as-a-judge evaluation.

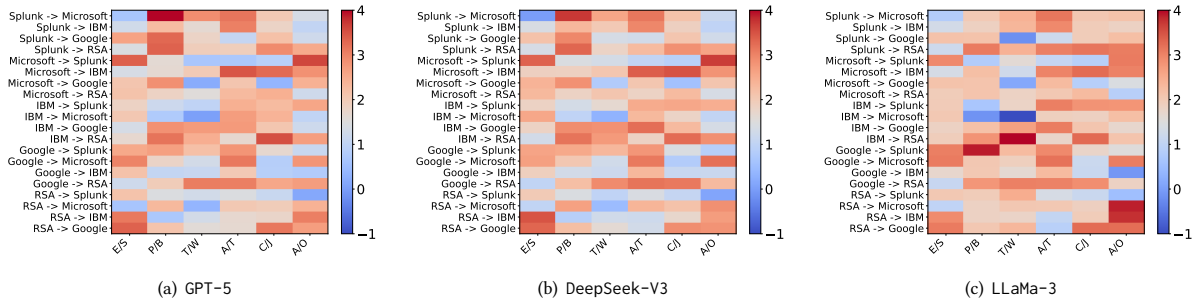


Figure 4: Fine-grained conversion evaluation on semantic-level: redder regions highlight systematic improvements, and blue cells indicate challenges in schema mapping.

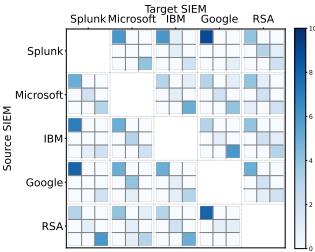


Figure 5: Consistency between LLM-as-a-Judge and human evaluation for heterogeneous SIEM rule conversions. Each off-diagonal cell corresponds to one source–target conversion and contains a 3×3 confusion matrix: rows are human judgments (ARULECON better / Tie / Baseline better), and columns are LLM judgments. Diagonal entries show agreement, while off-diagonal entries show mismatches.

Results. The radar chart in Figure 6 illustrates the results of the LLM-based evaluator. This plots reveal that ARULECON consistently surpasses the baseline across all six semantic facets, with the most pronounced gains in *Predicate & Boolean Logic* and *Alert Semantics/Outcome*. GPT-5 (Figure 6a) demonstrates the most balanced and stable improvements, DeepSeek (Figure 6c) achieves noticeable gains but with smaller margins in *Scope/Schema* and *Temporal*, while LLaMA (Figure 6e) shows narrower coverage and occasional regressions, especially in *Aggregation & Thresholding*. We show every source-target cases in Figure 4, which further confirm these observations: conversions involving Splunk and RSA NetWitness preserves more semantic validity. Overall, these results validate that ARULECON achieves consistent semantic fidelity across heterogeneous SIEMs and remains effective regardless of the underlying backbone LLM.

Ablation Study. Figure 7 shows the semantic evaluation from the LLM-as-a-judge. The complete ARULECON consistently outperforms its ablated variants across all six dimensions. Removing IR mainly reduces performance on *Aggregation & Thresholding* (A/T) and *Correlation & Joins* (C/J), since these require explicit step decomposition. Without Agentic RAG, scores on *Predicate & Boolean Logic* (P/B) and *Event Scope & Schema Mapping* (E/S) drop more, likely due to weaker field alignment and operator mapping. Excluding Python checks most affects *Alert Semantics & Outcome* (A/O),

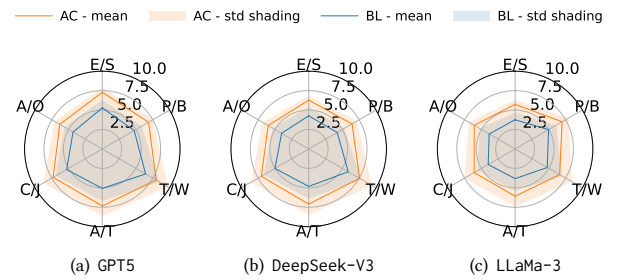


Figure 6: Semantic evaluation of ARULECON.

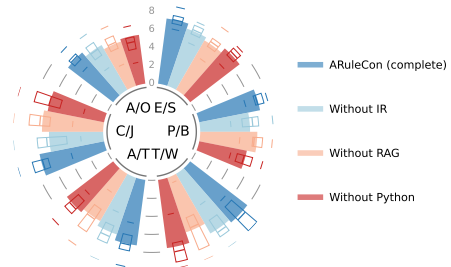


Figure 7: Ablation study of semantic metrics on ARULECON.

as execution mismatches cannot be detected. Overall, the results indicate that each component contributes complementary strengths, and their combination is crucial for robust semantic consistency.

C.3 Case Analysis

Nested Query Structures. A common difficulty in cross-SIEM rule translation emerges when rules contain nested query logic. Splunk allows such patterns through subsearches (`[...]`), enabling analysts to embed one query as the filter condition of another. Consider the following detection task: identify outbound network flows to rare non-US destinations, but only for source IPs that have already triggered multiple failed logins within a recent time window. This is to detect the realistic attack pattern when adversaries attempt brute-force authentication before establishing external connections. In Splunk SPL, this can be expressed with a nested query:

```
index=netflow action=allowed dest_country!=US
| lookup geoip ip as dest_ip output country as
  dest_country
```

```
| where dest_country != "US"
[ search index=auth action=failure user!=root earliest
  =-30m
  | stats count as fail_cnt by src_ip
  | where fail_cnt >= 5
  | fields src_ip ]
| stats dc(dest_country) as unique_countries, values(
  dest_country) as dest_list by src_ip
| where unique_countries >= 2
```

The nested form in Splunk is challenging to be directly mapped to another SIEM-sules, which lacks subsearch support. Chronicle requires each event type to be explicitly declared and joined, with conditions written as correlations over event variables. Through the use of IR, the nested logic can be decomposed into a series of sequential steps that separate the construction of suspicious IP sets from the subsequent flow analysis. For the case of outbound connections from IPs with repeated failed logins, the IR representation is as follows:

```
[
/* ... */
{
  "keyword": "failed_login_count",
  "param": "src_ip >= 5 within 30m",
  "description": "Identify source IPs with at least 5
    ↪ failed logins in the last 30 minutes"
},
{
  "keyword": "outbound_flow_filter",
  "param": "dest_country != US",
  "description": "Select outbound connections to non-US
    ↪ destinations for those IPs"
},
/* ... */
]
```

Based on this structured breakdown, the corresponding Chronicle YARA-L rule can then be written as:

```
rule brute_force_outbound {
  meta:
    description = "Outbound connections to non-US from
      IPs with >= 5 failed logins in 30m"
  events:
    $fail e1 = m.system.auth {
      filter: security_result.action = "BLOCK"
    }
    $flow e2 = net.flow {
      filter: network.connection_action = "ALLOW"
      and destination.country != "US"
    }
  match:
    $fail.src_ip = $flow.src_ip over 30m
  condition:
    count($fail) by $fail.src_ip >= 5
    and count_distinct($flow.destination.country) >= 1
  outcome:
    $fail.src_ip,
    $flow.destination.country
}
```

Temporal Windows & Thresholds. Temporal windows and thresholds are widely used in SIEM rules, for example, to detect repeated failed login attempts within a fixed time range. Different SIEMs implement the semantics with different syntactic constructs. Splunk

discretizes time with `bucket span=...` and then aggregates via `stats` (e.g., `| bucket _time span=10m | stats count by ... , _time`). Microsoft Sentinel (KQL) instead bins timestamps with `bin(TimeGenerated, ...)` inside `summarize` (e.g., `... | summarize count() by ..., bin(TimeGenerated, 10m)`). Beyond keyword differences (`bucket` vs. `bin`), they also diverge on time column names (`_time` vs. `TimeGenerated`) and the placement of filters relative to aggregation.

Consider the following Splunk SPL rule, which detects potential DNS tunneling behavior. The rule flags any host that, within a 10-minute window, issues at least 100 DNS to rare top-level domains (such as .xyz, .pw), across three or more distinct second-level domains.

```
index=dns sourcetype=dns:query
| eval tld=lower(replace(query, ".*\.\.([A-Za-z0-9-]+)$",
  "\1"))
| where tld IN ("xyz","pw","top")
| eval sld=lower(replace(query, ".*\.\.([A-Za-z0-9-]+)\.([A-
  Za-z0-9-]+)$", "\1"))
| bucket _time span=10m
| stats count as qps, dc(sld) as sld_cnt, values(sld) as
  sld_set by host, _time
| where qps >= 100 AND sld_cnt >= 3
```

Through Agentic RAG, the system simulates how an analyst would interact with vendor documentation during translation. From the IR intent, the agent first generates exploratory keywords such as “KQL time-window aggregation” and “KQL distinct count”. The retrieval step then surfaces the official `summarize` operator page, which prescribes the use of `summarize ... by bin(TimeGenerated, 10m)`, and the list of supported aggregation functions including `dcount()` and `make_set()`. From these references, the agent acquires concrete knowledge: Splunk’s `bucket _time span=30m` should be rewritten as `bin(TimeGenerated, 30m)`, `dc()` maps to `dcount()`. The faithful KQL rule produced through this process is shown below.

```
DnsEvents
| extend TLD = tolower(extract(@".*\.\.([A-Za-z0-9-]+)$",
  1, QueryName))
| where TLD in ("xyz","pw","top")
| extend SLD = tolower(extract(@".*\.\.([A-Za-z0-9-]+)\.([A-
  Za-z0-9-]+)$", 1, QueryName))
| summarize qps = count(),
  sld_cnt = dcount(SLD),
  sld_set = make_set(SLD)
  by Computer, bin(TimeGenerated, 10m)
| where qps >= 100 and sld_cnt >= 3
```